Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computación

Análisis y Procesamiento OLAP

Diseño General del Sistema

Camila Thompson Sancho

I Semestre
Año lectivo 2026

Curso de Bases de Datos II
Profesor: Kenneth Obando Rodriguez

Jueves 25 de junio de 2026

Tabla de contenidos

1. Introducción **2**
2. Arquitectura **2**
2.1. Lógica 2
2.1. Física 3
3. Microservicios **5**
4. Flujo de Datos **8**
4.1. Flujo Transaccional 8
4.2. Flujo Analítico 9
5. Esquema del Data Warehouse **12**
5.1. Esquema Estrella 12
5.1. Cubos OLAP 14
6. Modelo de Grafo **14**
6.1. Construcción del grafo 15
6.2. Datos simulados 16
6.3. Consultas Cypher de análisis 17
Co-compra: productos comprados juntos con mayor frecuencia 17
Usuarios influyentes: quién recomienda a quién 17
Caminos mínimos entre ubicaciones 18
7. Asignación de Rutas de Entrega **18**
7.1. Consultas Cypher de análisis 18
7.2. Consultas Cypher de ruteo 19
Ruta completa de un repartidor 19
Carga de trabajo por repartidor 19
Asignación en tiempo real para un pedido nuevo 19

1. Introducción

El presente documento detalla la estructura técnica y el funcionamiento de un sistema de gestión de restaurantes, diseñado bajo un modelo de microservicios contenerizados. La primera fase del proyecto cubrió la operación transaccional del negocio: el flujo de reservaciones, la búsqueda de productos y la seguridad de los usuarios, con capacidad de adaptación ante variaciones en el motor de base de datos.

Esta segunda fase extiende el sistema con una capa analítica que consolida los datos transaccionales en un data warehouse, automatiza su actualización periódica, los expone mediante dashboards de negocio, y modela las relaciones entre usuarios, pedidos y repartidores como un grafo para análisis de patrones de consumo y optimización de rutas de entrega. Esta capa se compone de cinco tecnologías nuevas que se describen en detalle en las secciones siguientes, incluyendo los comandos exactos para desplegarlas, las decisiones de diseño tomadas y ejemplos reales de uso.

2. Arquitectura

En esta sección se presentan las especificaciones del sistema y la arquitectura de microservicios utilizada. El diseño se aborda desde dos perspectivas complementarias, la interacción entre los servicios y cómo estos fueron implementados específicamente para este giro de negocio.

2.1. Lógica

La arquitectura lógica del sistema transaccional se mantiene sin cambios respecto a la primera fase: una capa de abstracción (DAO) que permite operar sin importar el motor de base de datos, una capa de caché para reducir la latencia, y un servicio de autenticación para centralizar la seguridad.

A esta arquitectura se añade una capa analítica que opera de forma desacoplada y asíncrona respecto al flujo transaccional. En lugar de consultar las

bases de datos operacionales en cada solicitud, la capa analítica extrae los datos periódicamente, los transforma, y los deja disponibles en un almacén separado optimizado para consultas agregadas.

El flujo lógico de la capa analítica sigue cuatro etapas:

1. **Extracción:** lectura de los datos crudos desde MongoDB o PostgreSQL, según el que esté activo.
2. **Transformación:** Apache Spark limpia, agrega y reestructura los datos en memoria distribuida.
3. **Carga:** los resultados persisten como tablas permanentes en Apache Hive, organizadas tanto en forma de reportes agregados (cubos OLAP) como en un esquema dimensional (esquema estrella).
4. **Consumo:** Redash consulta Hive vía JDBC para construir dashboards, mientras que Neo4j mantiene un modelo de grafo independiente, construido directamente desde las fuentes transaccionales, para análisis de relaciones y rutas.

Todo el ciclo de extracción, transformación y carga se ejecuta automáticamente cada semana mediante Apache Airflow, que además decide si es necesario reindexar el motor de búsqueda (Elasticsearch) cuando detecta cambios en el catálogo de productos.

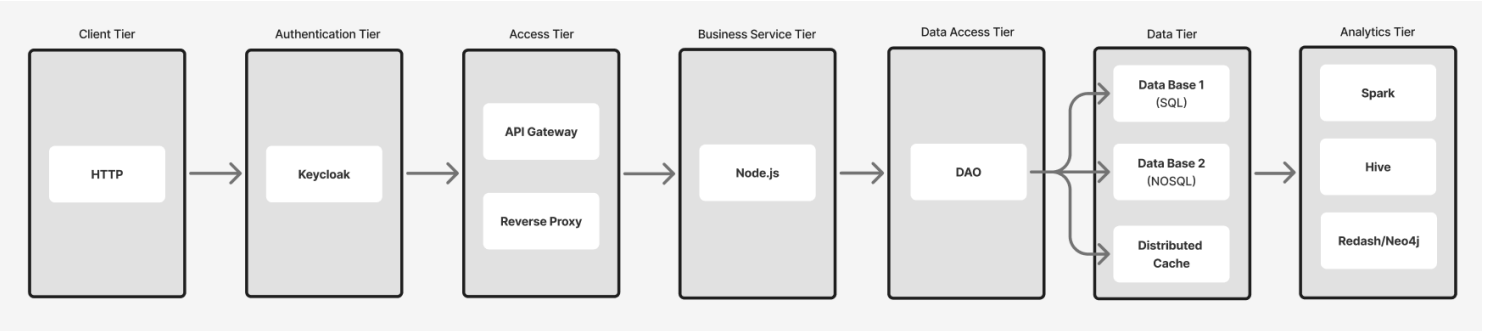


Figura 1. Diagrama de Arquitectura Lógica

2.1. Física

La implementación física incorpora una capa analítica independiente de la infraestructura transaccional. Ambas comparten la misma red Docker (olap-network) para acceder a los datos, pero se gestionan mediante stacks de Docker Compose separados, permitiendo activar o desactivar los servicios analíticos sin afectar la operación del sistema transaccional. Esta separación facilita la administración de recursos debido al alto consumo de memoria de herramientas como Spark, Airflow y Neo4j.

El stack analítico se compone de tres grupos de contenedores:

- **Procesamiento y almacenamiento:** agrupa Spark (Master, Worker y un contenedor Jupyter para ejecutar las transformaciones), Hive (Metastore y HiveServer2, cada uno con su base PostgreSQL interna de metadatos) y Redash (servidor, worker, scheduler, con su propia base PostgreSQL y cola Redis). Se agrupan juntos porque Redash depende directamente de que Hive esté disponible para poder construir sus dashboards.
- **Orquestación:** Airflow (webserver, scheduler, worker, con su propia base PostgreSQL y cola Redis). Se mantiene separado porque orquesta al resto de los servicios mediante llamadas externas (docker exec), no requiere estar siempre activo, y tiene un costo de memoria considerable que conviene apagar cuando no se está corriendo el DAG.
- **Grafos:** Neo4j en solitario. Se mantiene separado porque no depende de Hive ni de Spark: el grafo se construye directamente desde las fuentes transaccionales (MongoDB/PostgreSQL) mediante un script Python independiente, sin pasar por el pipeline de Spark.

A continuación se presenta el diagrama de arquitectura física, donde se distinguen el stack transaccional original y el stack analítico:

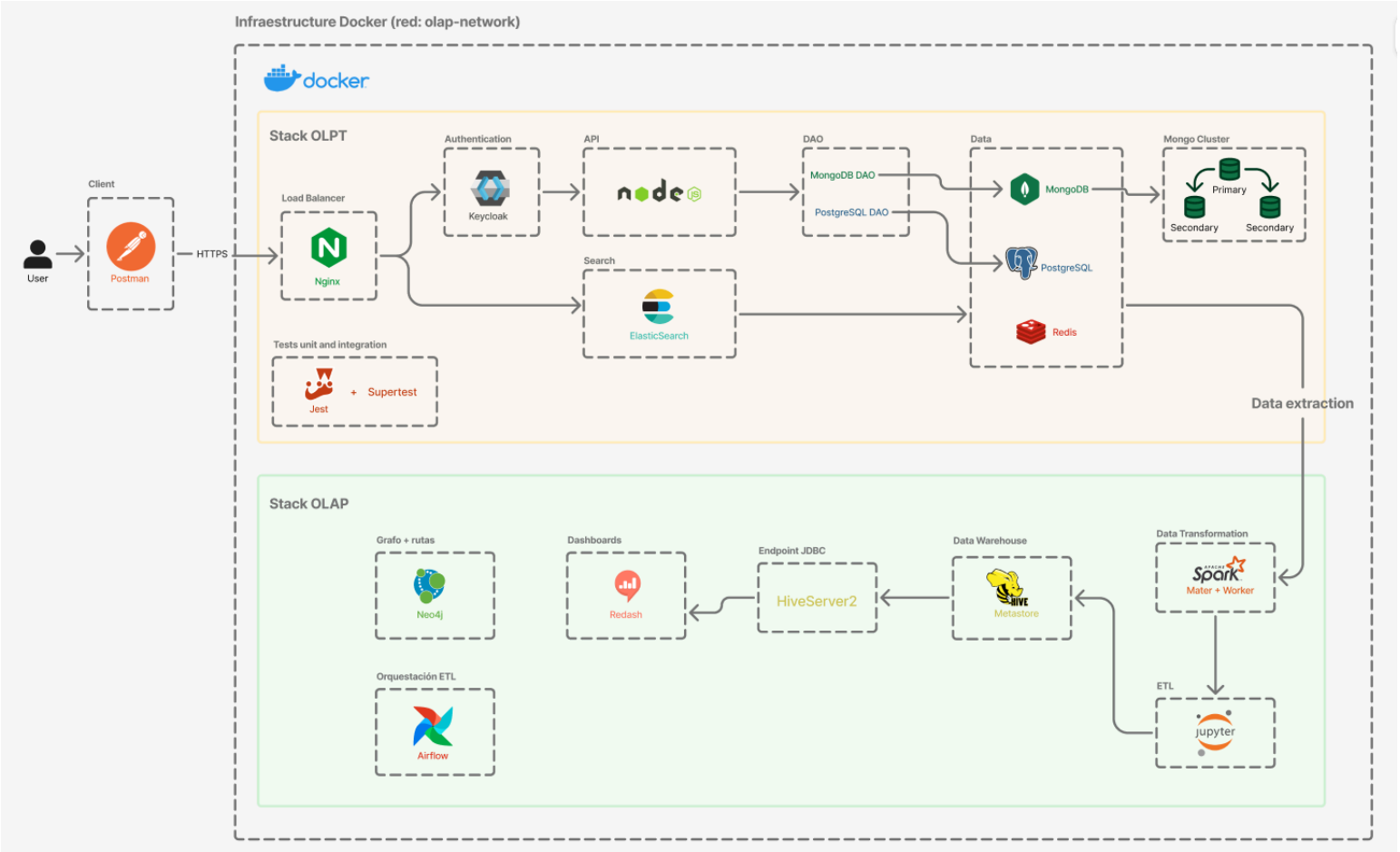


Figura 2. Diagrama de Arquitectura Física

3. Microservicios

En esta sección se detalla el propósito de cada componente (contenedor) dentro del ecosistema del sistema. Los primeros seis microservicios corresponden a la fase transaccional original; a continuación se documentan los cinco componentes nuevos que conforman la capa analítica, manteniendo el mismo nivel de detalle pero incorporando además los puertos de acceso, comandos de verificación y las decisiones de diseño tomadas durante su implementación.

- **API Gateway / Balanceador (Nginx):** Es el punto de entrada único al sistema. Se encarga de recibir todas las peticiones de los clientes, gestionar el tráfico de red y distribuirlo de manera eficiente hacia las instancias del backend para evitar cuellos de botella.

- **Servicio de Identidad (Keycloak):** Responsable de la seguridad perimetral. Administra el ciclo de vida de los usuarios, la emisión de tokens JWT y la validación de roles, asegurando que solo personal autorizado (administradores o clientes) acceda a funciones específicas.
- **Servicio Backend (Node.js):** Contiene la lógica de negocio del restaurante. Se comunica con las capas de persistencia y caché para procesar cada solicitud.
- **Motor de Búsqueda (Elasticsearch):** Microservicio especializado en la indexación y recuperación rápida de información. Su responsabilidad es permitir búsquedas para los platos ya sea por nombre o categorías.
- **Capa de Caché (Redis):** Su responsabilidad es almacenar temporalmente los datos de consulta frecuente para responder al usuario sin necesidad de consultar las bases de datos principales en cada ocasión.
- **Gestores de Persistencia (PostgreSQL / MongoDB):** Responsables del almacenamiento permanente.
    - PostgreSQL: Maneja datos estructurados y relaciones complejas.
    - MongoDB: Gestiona grandes volúmenes de datos mediante sharding y asegura la disponibilidad mediante su esquema de réplicas.
- **Motor de Procesamiento Distribuido (Apache Spark):** Ejecuta las transformaciones que alimentan los reportes OLAP y el esquema estrella. Corre en modo cluster (spark-master, UI en el puerto 8080; uno o más spark-worker), y se accede de forma interactiva mediante un contenedor Jupyter (puerto 8888), donde los notebooks leen las fuentes, transforman con PySpark, y escriben en Hive.
- **Data Warehouse (Apache Hive):** Almacena de forma permanente los resultados de los análisis OLAP y las tablas del esquema estrella. Se compone de hive-metastore (catálogo de tablas, con su base PostgreSQL interna) y hiveserver2 (endpoint JDBC en el puerto 10000, consumido por Redash). Los datos físicos (Parquet) viven en un volumen Docker compartido con Spark.
- **Visualización de Datos (Redash):** Construye los dashboards de negocio mediante consultas SQL contra Hive vía JDBC (puerto 5000 para la interfaz web). Las consultas apuntan a los cubos OLAP pre-calculados (vistas en

Hive, ver Sección 5) en lugar de a las tablas base, ya que un JOIN directo sobre hechos y dimensiones excede el timeout por defecto de Redash.

- **Orquestador de Flujos (Apache Airflow):** Automatiza semanalmente el pipeline de extracción, transformación, carga y reindexado condicional (interfaz en el puerto 8090). Las tareas que requieren Spark o Hive no instalan esas herramientas dentro de Airflow; en su lugar, ejecutan comandos sobre los contenedores correspondientes (docker exec/docker cp hacia spark-master y hiveserver2), reutilizando los servicios ya desplegados. Esto requiere que el worker de Airflow tenga acceso al socket de Docker del host.
- **Motor de Grafos (Neo4j):** Modela las relaciones entre usuarios, pedidos, platos, restaurantes y repartidores (interfaz web en el puerto 7474, protocolo Bolt en el 7687). A diferencia de Redash, el grafo no se construye desde Hive sino directamente desde MongoDB mediante un script Python independiente, porque requiere el detalle a nivel de documento (qué platos contiene cada pedido) que se pierde al agregar en los cubos.

A continuación se resumen los microservicios del sistema transaccional y la capa analítica:

Microservicio	Función	Acceso	Stack
Nginx	Balanceador de carga; punto de entrada único al sistema	HTTP: 80	docker-compose.yml
Keycloak	Gestión de identidad, emisión de tokens JWT y validación de roles	UI/API: 8081	docker-compose.yml
Node.js API	Lógica de negocio del restaurante (reservas, menús, mesas)	API: 5001	docker-compose.yml
Elasticsearch	Indexación y búsqueda de platos por nombre o categoría	API: 9200	docker-compose.yml

Redis	Caché de consultas frecuentes para reducir latencia	Interno: 6379	docker-compose.yml
MongoDB	Persistencia de datos crudos (modo NoSQL, con sharding y réplicas)	27017	docker-compose.yml
PostgreSQL	Persistencia de datos crudos (modo relacional)	4001	docker-compose.yml

Tabla 1. Resumen de microservicios de sistema transaccional

Microservicio	Función	Acceso	Stack
Apache Spark	Procesa y transforma los datos (OLAP + esquema estrella)	UI cluster: 8080 Jupyter: 8888	docker-compose_olap.yml
Apache Hive	Almacena el data warehouse de forma permanente	JDBC: 10000 UI diagnóstico: 10002	docker-compose_olap.yml
Redash	Construye dashboards de negocio sobre Hive	UI web: 5000	docker-compose_olap.yml
Apache Airflow	Orquesta el pipeline ETL semanal de forma automática	UI web: 8090	docker-compose_airflow.yml
Neo4j	Modela usuarios, pedidos y repartidores como grafo	UI Browser: 7474 Bolt: 7687	docker-compose_neo4j.yml

Tabla 2. Resumen de microservicios de capa analítica

4. Flujo de Datos

El flujo de información en el sistema sigue dos trayectorias distintas: el flujo transaccional, que atiende las solicitudes de los usuarios en tiempo real, y el flujo

analítico, que se ejecuta de forma periódica y desacoplada para mantener actualizado el data warehouse, los dashboards y el grafo de relaciones.

4.1. Flujo Transaccional

1. **Entrada y Distribución:** Cuando un usuario realiza una acción la petición llega al balanceador Nginx. Este redirige la solicitud al contenedor de la API o del servicio de búsqueda.
2. **Validación de Seguridad:** El backend recibe la petición y, antes de procesarla, verifica con Keycloak que el usuario tenga una sesión activa y los permisos necesarios para realizar la solicitud.
3. **Consulta de Optimización:** El sistema primero pregunta a Redis si la información solicitada ya fue consultada recientemente.
    - Si existe (Cache Hit): Los datos se devuelven inmediatamente.
    - Si no existe (Cache Miss): Se procede a consultar la base de datos correspondiente.
4. **Acceso a Datos y Búsqueda:**
    - Si el usuario está buscando un plato específico por texto, el flujo se dirige a Elasticsearch.
    - Si es una transacción (como realizar una reserva), el flujo se dirige a PostgreSQL o MongoDB (utilizando el DAO para decidir la ruta).
5. **Cierre del Ciclo:** Una vez obtenidos los datos, el backend procesa la respuesta, actualiza el caché en Redis si es necesario, y envía la información de vuelta al usuario a través del balanceador.

4.2. Flujo Analítico

Este flujo no responde a una solicitud de usuario, sino que se ejecuta automáticamente cada semana, disparado por Airflow.

1. **Disparo:** El scheduler de Airflow activa el DAG etl_restaurantes según su calendario (schedule_interval="@weekly"), o un usuario lo dispara manualmente desde la UI (Trigger DAG) indicando la fuente de datos a usar mediante un parámetro JSON: {"fuente": "mongo"} o {"fuente": "postgres"}.

Este parámetro se propaga a todas las tareas siguientes mediante el mecanismo de params de Airflow.

2. **Extracción:** Esta tarea corre código Python directamente dentro del contenedor airflow-worker (no requiere tocar otros contenedores). Según el valor de fuente, abre una conexión con pymongo o psycopg2 y ejecuta una operación mínima de verificación: en MongoDB, list_collection_names(); en PostgreSQL, SELECT COUNT(*) FROM pedidos. Si la conexión falla o no responde en 5 segundos, la tarea falla y el DAG se detiene aquí, no tiene sentido continuar si la fuente no está disponible.
3. **Transformación:** Esta es la tarea más compleja del flujo y la única que requiere comunicación entre contenedores. El procedimiento exacto es:
    a. La tarea construye, como una cadena de texto Python, el script completo de PySpark (las funciones de los tres análisis OLAP y de las siete tablas del esquema estrella, parametrizadas según fuente).
    b. Escribe ese texto en un archivo temporal dentro del propio contenedor de Airflow.
    c. Copia ese archivo al contenedor spark-master mediante docker cp archivo_temporal.py spark-master:/tmp/etl_airflow.py.
    d. Ejecuta el script dentro de spark-master mediante docker exec spark-master/opt/spark/bin/spark-submit--masterspark://spark-master:7077 /tmp/etl_airflow.py.
    e. Dentro de ese script, Spark lee las colecciones o tablas de origen, calcula los tres análisis (tendencias de consumo, horarios pico, crecimiento mensual) y construye las cinco dimensiones y dos tablas de hechos del esquema estrella, guardando cada resultado con DataFrame.write.saveAsTable() directamente en Hive.
4. **Carga (tarea cargar_hive):** Para confirmar que la escritura anterior fue exitosa, esta tarea ejecuta, también mediante docker exec, el cliente de línea de comandos beeline dentro del contenedor hiveserver2, para cada una de las seis tablas que el paso anterior debió crear. Si alguna consulta falla o no retorna un conteo, la tarea falla. El mismo principio de no instalar el cliente dentro de Airflow se aplica aquí: en vez de instalar pyhive en el contenedor de Airflow, se reutiliza el cliente que ya existe dentro de hiveserver2.

5. **Verificación de catálogo:** Esta tarea obtiene dos conjuntos de identificadores: los IDs de la colección/tabla platos en la fuente transaccional (consultando Mongo o Postgres directamente), y los IDs actualmente indexados en Elasticsearch. Compara ambos conjuntos con una resta de conjuntos en Python. El resultado de esta comparación determina el valor de retorno de la tarea, que en un BranchPythonOperator no es un dato sino el nombre de la siguiente tarea a ejecutar: "reindexar_elasticsearch" si hubo cambios, "sin_cambios_es" si no los hubo. Airflow ejecuta únicamente la rama elegida y marca la otra como skipped (no como fallida).
6. **Reindexado condicional:** Si se llegó a esta tarea, se reconstruye el índice completo: primero se elimina el índice platos existente (DELETE /platos), luego se reconstruye desde cero con una operación bulk (POST /_bulk) que inserta todos los documentos actuales de la fuente. Se opta por una reconstrucción completa y no por una actualización incremental porque el catálogo es pequeño y esto evita tener que rastrear altas, bajas y modificaciones por separado.
7. **Consumo:** Con el data warehouse actualizado, Redash no requiere ninguna acción adicional: sus consultas SQL apuntan directamente a las tablas y vistas (cubos) de Hive, por lo que la próxima vez que un dashboard se recargue o el scheduler de Redash lo refresque, reflejará automáticamente los datos recién cargados.
8. **Actualización del grafo (independiente del DAG):** La actualización de Neo4j no forma parte del DAG de Airflow y debe ejecutarse manualmente mediante el script **build_graph.py**. Esta decisión se debe a que el grafo se reconstruye completamente en cada ejecución y cumple un propósito distinto, el grafo se utiliza para análisis de relaciones y rutas, por lo que ambos procesos se mantienen desacoplados.

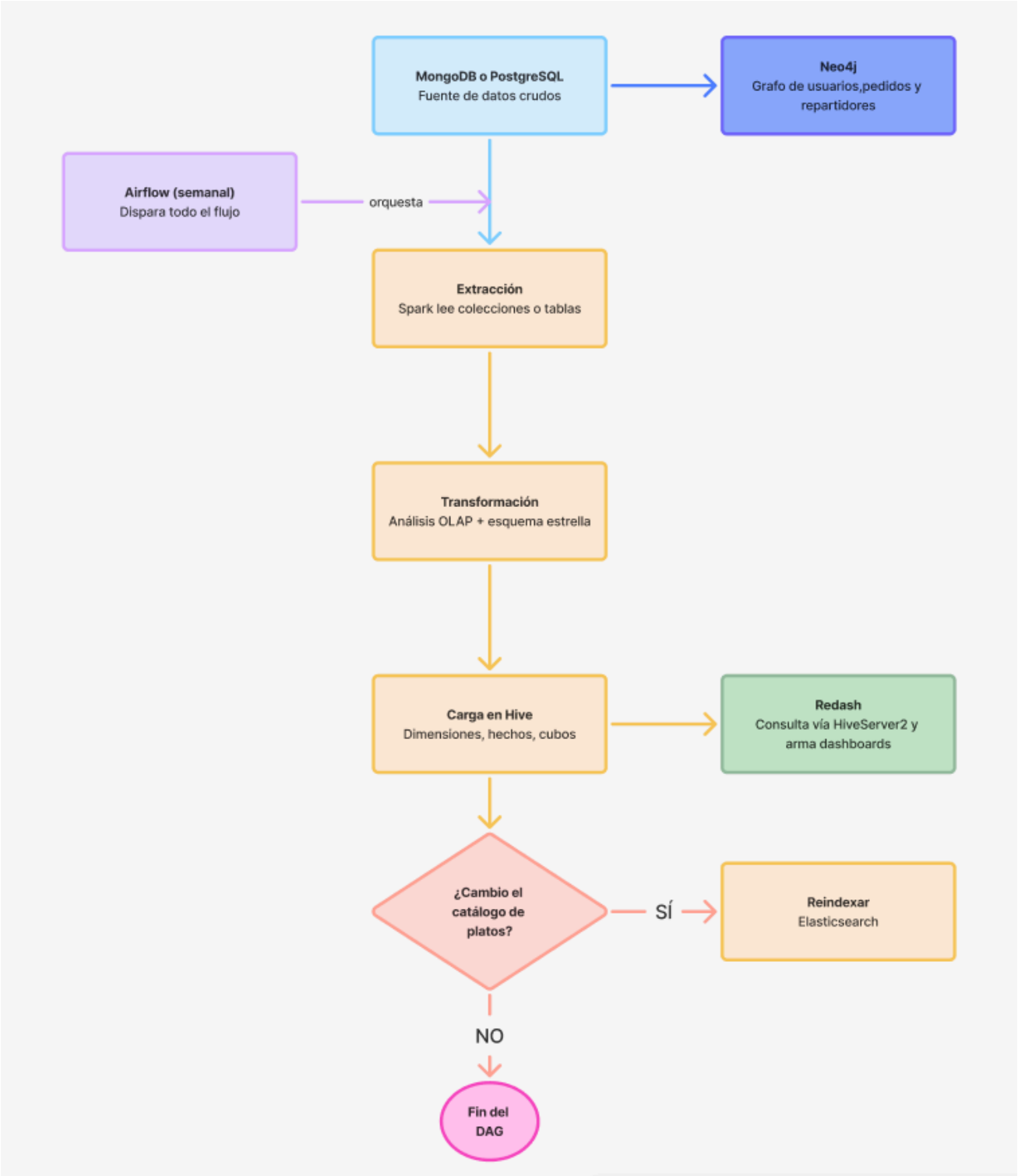


Figura 3. Diagrama de flujo

El grafo de Neo4j sigue un ciclo independiente y manual: no está integrado al DAG de Airflow. Para reflejar el estado más reciente de los datos transaccionales en el grafo, es necesario ejecutar manualmente python build_graph.py.

5. Esquema del Data Warehouse

El almacén de datos en Hive se organiza en dos bases independientes: “restaurantes”, que contiene los tres reportes OLAP calculados directamente sobre los datos crudos, y “estrella”, que contiene el modelo dimensional (esquema estrella) construido para soportar consultas analíticas flexibles sin depender de reportes predefinidos.

5.1. Esquema Estrella

El esquema estrella consolida los datos históricos de pedidos, reservaciones, productos y usuarios en dos tablas de hechos y cinco dimensiones compartidas:

Tabla	Tipo	Contenido
fact_pedidos	Hechos	Un registro por cada ítem dentro de un pedido: cantidad, precio total, estado
fact_reservaciones	Hechos	Un registro por reservación: hora, estado
dim_usuario	Dimensión	Identificador, nombre, email
dim_plato	Dimensión	Nombre, precio, categoría (heredada del menú)
dim_mesa	Dimensión	Número, capacidad, restaurante al que pertenece
dim_restaurante	Dimensión	Nombre, ubicación, contacto
dim_tiempo	Dimensión	Fecha desglosada en año, mes, día y día de la semana

Tabla 3. Tablas creadas en Hive

Ambas tablas de hechos fact_pedidos y fact_reservaciones se relacionan con dim_tiempo para análisis temporales y con dim_usuario para analizar el comportamiento de los clientes. Además, fact_pedidos se conecta con dim_plato, mientras que fact_reservaciones se relaciona con dim_mesa y, a través de esta, con dim_restaurante.

La dimensión dim_tiempo normaliza los distintos formatos de fecha provenientes de MongoDB y PostgreSQL a un único tipo de fecha, permitiendo generar de forma consistente atributos como año, mes y día de la semana, independientemente de la fuente de origen.

Seguidamente se presenta el diagrama correspondiente a este esquema estrella:

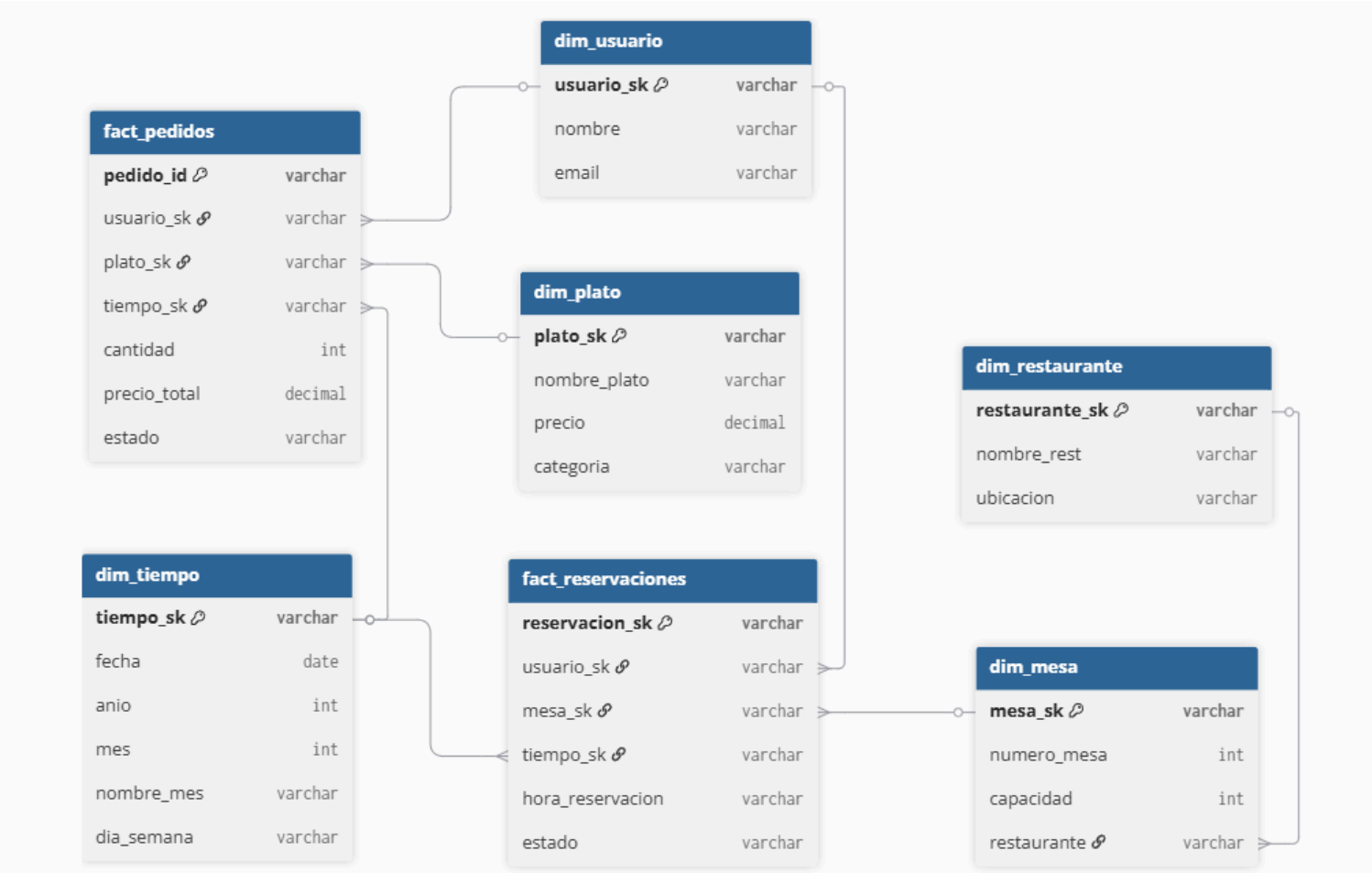


Figura 4. Diagrama ERD del esquema estrella

5.1. Cubos OLAP

Sobre el esquema estrella se construyeron cinco vistas materializadas (cubos OLAP) que pre-calculan las agregaciones más consultadas, permitiendo que Redash las consuma sin necesidad de ejecutar JOINs costosos en cada solicitud:

Cubo	Agrupa por	Responde
cubo_ventas_por_tiempo	Año, mes	Ingresos y cantidad de pedidos por período
cubo_ventas_por_ubicacion	Restaurante, ubicación	Ingresos por restaurante
cubo_ventas_por_producto	Categoría, plato	Platos más vendidos y su contribución a ingresos
cubo_frecuencia_usuarios	Usuario	Cantidad de pedidos, gasto total y última actividad por cliente
cubo_reservaciones_por_tiempo	Año, mes, hora, restaurante	Volumen de reservaciones y tasa de finalización vs. cancelación

Tabla 4. Cubos creados

Los cubos se implementan como tablas físicas en lugar de vistas debido a una incompatibilidad encontrada entre la versión del Metastore de Hive y el motor de ejecución de Spark. Esta solución evita errores de ejecución y mejora el rendimiento de consulta en Redash, aunque los datos solo se actualizan cuando se ejecuta nuevamente el pipeline, lo cual es adecuado al realizarse de forma semanal.

6. Modelo de Grafo

Mientras que el data warehouse en Hive se construye para responder preguntas agregadas (cuánto se vendió, en qué mes, en qué restaurante), el grafo en Neo4j se construye para responder preguntas sobre relaciones individuales: qué platos se compran juntos, qué usuarios influyen sobre otros, y cómo moverse eficientemente entre puntos geográficos. Por esta razón, el grafo no se deriva del data warehouse sino que se construye directamente desde las fuentes transaccionales (MongoDB o PostgreSQL).

6.1. Construcción del grafo

El script se conecta a MongoDB y a Neo4j y reconstruye el grafo completo desde cero en cada ejecución: primero elimina todos los nodos y relaciones existentes, luego recrea cada tipo de nodo y relación en el orden necesario para que las referencias entre ellos puedan resolverse.

El grafo modela cinco tipos de nodo:

Nodo	Origen de los datos
Usuario	Identificadores reales encontrados en pedidos.usuario_id
Restaurante	Colección restaurantes, con coordenadas simuladas dentro de Costa Rica
Plato	Colección platos, vinculado a su restaurante a través de menús
Pedido	Colección pedidos
Repartidor	Simulado (ocho nodos con ubicación inicial simulada dentro de Costa Rica)

Tabla 5. Nodos modelados en el grafo

Sobre estos nodos se construyen seis tipos de relación:

Relación	Dirección	Significado
HIZO	Usuario → Pedido	Quién realizó cada pedido
CONTIENE	Pedido → Plato	Qué platos incluye un pedido, con la cantidad como propiedad
PERTENECE_A	Plato → Restaurante	A qué restaurante pertenece cada plato
RECOMIENDA	Usuario → Usuario	Simulada
CERCA_DE	Restaurante ↔ Restaurante	Distancia real entre restaurantes a 50 km o menos
ENTREGA	Repartidor → Pedido	Asignación de un repartidor a un pedido

Tabla 6. Relaciones entre nodos

A continuación se presenta el diagrama del modelo de grafo creado:

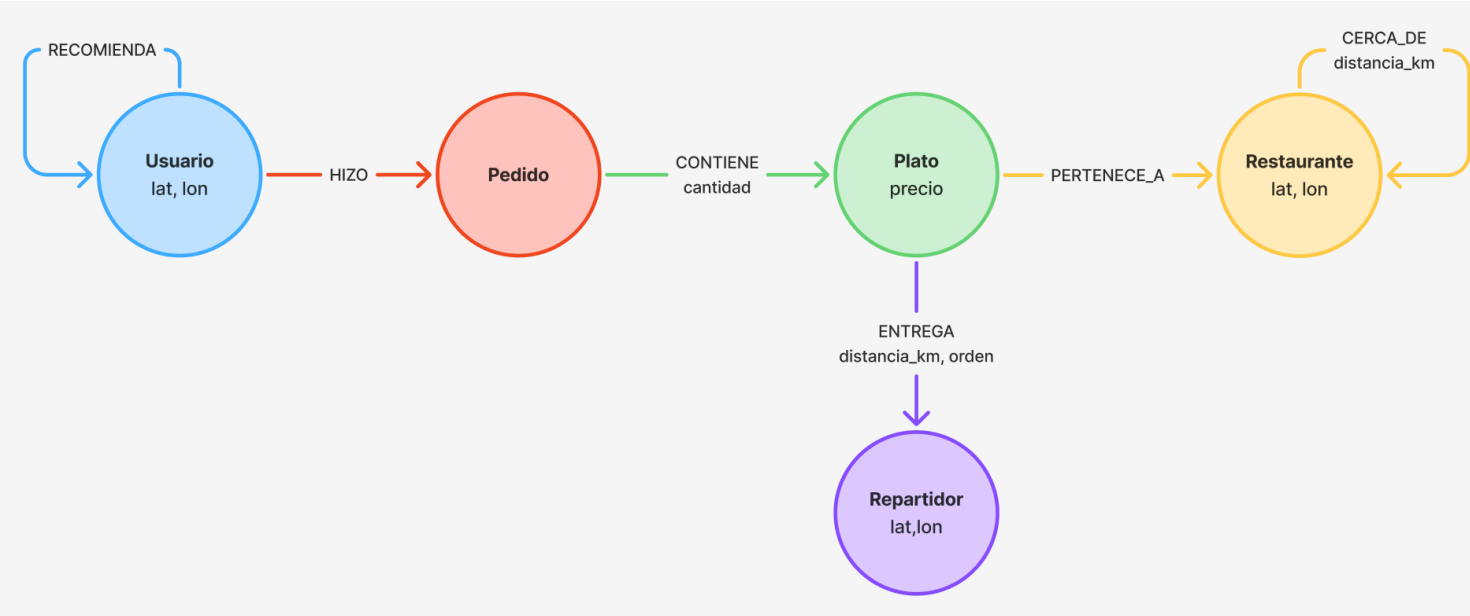


Figura 5. Diagrama del modelo de grafo

6.2. Datos simulados

Debido a que el sistema transaccional no almacena información geográfica ni relaciones de recomendación entre usuarios, algunos elementos del grafo se generan de manera simulada y reproducible. Estas relaciones permiten realizar análisis espaciales y de interacción entre usuarios dentro del modelo.

- **Coordenadas geográficas:** A cada Usuario y Restaurante se le asignan coordenadas aleatorias dentro de Costa Rica para calcular distancias y apoyar la asignación de repartidores.
- **RECOMIENDA:** Se crean relaciones simuladas entre algunos usuarios, con una propiedad afinidad (0.5 a 1.0), para analizar influencia y recomendaciones entre usuarios.
- **CERCA_DE:** Se establece a partir de la distancia entre restaurantes calculada con sus coordenadas geográficas.

- **Cálculo de distancias:** Las distancias utilizadas en **CERCA_DE** y en la asignación de repartidores se calculan obteniendo la distancia en kilómetros entre dos puntos geográficos.

6.3. Consultas Cypher de análisis

Sobre este grafo se implementaron las siguientes consultas, agrupadas según el tipo de pregunta que responden:

Co-compra: productos comprados juntos con mayor frecuencia

Para cada pedido, se buscan todos los pares de platos que contiene y se cuenta en cuántos pedidos distintos aparece cada par:

```
MATCH (pe:Pedido)-[:CONTIENE]->(p1:Plato)
MATCH (pe)-[:CONTIENE]->(p2:Plato)
WHERE p1.id < p2.id
WITH p1.nombre AS plato1, p2.nombre AS plato2, count(DISTINCT pe) AS
veces_juntos
RETURN plato1, plato2, veces_juntos
ORDER BY veces_juntos DESC
LIMIT 5
```

La condición p1.id < p2.id evita contar cada par dos veces (una vez como A-B y otra como B-A).

Usuarios influyentes: quién recomienda a quién

Se obtiene el listado completo de relaciones de recomendación, y por separado se identifica a los usuarios más recomendados (mayor cantidad de relaciones entrantes):

```
MATCH (u:Usuario)<-[:RECOMIENDA]-(otro:Usuario)
RETURN u.nombre AS usuario, count(otro) AS veces_recomendado
ORDER BY veces_recomendado DESC
LIMIT 10
```

Caminos mínimos entre ubicaciones

Se utiliza la función nativa shortestPath de Neo4j sobre la relación CERCA_DE, sumando la propiedad distancia_km de cada tramo del camino encontrado:

```
MATCH (origen:Restaurante {nombre: "Patio Tico"}),
      (destino:Restaurante {nombre: "La Candela"})
MATCH path = shortestPath((origen)-[:CERCA_DE*]-(destino))
RETURN [n IN nodes(path) | n.nombre] AS ruta,
      reduce(dist = 0.0, r IN relationships(path) | dist +
      r.distancia_km) AS distancia_total_km
```

Dado que las coordenadas se simulan aleatoriamente en todo el territorio nacional y CERCA_DE solo conecta pares a 50 km o menos, el grafo de restaurantes puede contener componentes desconectados entre sí.

7. Asignación de Rutas de Entrega

Esta sección extiende el grafo de Neo4j con la simulación de un módulo de reparto: ocho repartidores con ubicación inicial, asignados a los quinientos pedidos existentes mediante una heurística simple de vecino más cercano, ejercitada mediante consultas Cypher que muestran tanto la ruta resultante como la posibilidad de asignar repartidores a pedidos nuevos en tiempo real.

7.1. Consultas Cypher de análisis

La asignación de pedidos a repartidores no se resuelve con un algoritmo de optimización global, sino con una heurística greedy de vecino más cercano, ejecutada una sola vez como parte de la construcción del grafo:

Funcionamiento de la asignación de repartidores

1. Los pedidos se procesan en orden cronológico.
2. Para cada pedido, se calcula la distancia entre el cliente y cada repartidor utilizando la fórmula de Haversine.
3. El pedido se asigna al repartidor más cercano, creando una relación **ENTREGA** que almacena la distancia recorrida y el orden de asignación.
4. Después de cada asignación, la ubicación del repartidor se actualiza al punto de entrega realizado.

Esta estrategia permite simular rutas de entrega coherentes, ya que cada repartidor parte desde la ubicación de su última entrega. Aunque no garantiza una ruta óptima ni considera capacidades de carga, es suficiente para demostrar el mecanismo de asignación geográfica dentro del grafo.

7.2. Consultas Cypher de ruteo

Ruta completa de un repartidor

Recupera, en orden, todas las entregas asignadas a un repartidor específico:

```
MATCH (r:Repartidor {id:
"REP-001"})-[e:ENTREGA]->(pe:Pedido)<-[:HIZO]-(u:Usuario)

RETURN r.nombre AS repartidor,
       e.orden AS orden_entrega,
       pe.id AS pedido,
       u.nombre AS cliente,
       e.distancia_km AS distancia_km
ORDER BY e.orden
```

Carga de trabajo por repartidor

Resume cuántas entregas y qué distancia total acumuló cada repartidor, útil para detectar desbalances en la asignación:

```
MATCH (r:Repartidor)-[e:ENTREGA]->(pe:Pedido)

RETURN r.nombre AS repartidor,
       count(pe) AS total_entregas,
       round(sum(e.distancia_km), 2) AS distancia_total_km
ORDER BY total_entregas DESC
```

Asignación en tiempo real para un pedido nuevo

A diferencia de las dos consultas anteriores, esta consulta resuelve el mismo cálculo de vecino más cercano de forma dinámica para una ubicación arbitraria, sin necesidad de volver a ejecutar el script completo de construcción del grafo:

```
MATCH (r:Repartidor)
WITH r, point({latitude: r.lat, longitude: r.lon}) AS pos_repartidor,
     point({latitude: 9.93, longitude: -84.08}) AS pos_pedido
RETURN r.nombre AS repartidor,
       round(point.distance(pos_repartidor, pos_pedido) / 1000, 2) AS
distancia_km
ORDER BY distancia_km ASC
LIMIT 1
```

Esta consulta utiliza las funciones espaciales nativas de Neo4j (point y point.distance) para calcular proximidad directamente en Cypher, sin necesidad de lógica externa. Aunque cumple el mismo propósito que la fórmula de Haversine usada en el script, demuestra que el cálculo puede realizarse completamente dentro de la base de datos, lo que resulta útil para escenarios de asignación en tiempo real.